

Ray Cast Vision Lab

A Real-Time 2D Ray Casting & Vector Vision Simulator

Submitted By: Dharmit Sathvara

Roll Number: 150096725080

Institution: ITM Skills University

Subject: React Development

Batch: 2025–29

GitHub: Your GitHub Repository

Live Demo: Your Vercel Link

Problem Statement

RayCast Vision Lab is a browser-based optics and computational geometry simulator designed to visualize how light rays interact with obstacles in a two-dimensional environment.

The system addresses the following real-world challenges:

1. Visibility Analysis

Understanding how objects obstruct vision is difficult without visual simulation tools. The system provides real-time visibility calculations using ray casting.

2. Ray-Wall Intersection Detection

Every projected ray must accurately detect collisions against environmental boundaries using mathematical intersection equations.

3. Dynamic Field of View Simulation

Users need the ability to simulate both narrow directional vision and complete 360° visibility zones.

4. Interactive Environment Creation

Custom obstacle layouts must be created directly inside the simulation workspace without external tools.

5. Real-Time Rendering

The simulator must continuously update hundreds of rays while maintaining smooth frame rates.

6. Environment Persistence

Custom wall layouts and simulator settings should remain available after page refreshes.

7. Optical Performance Monitoring

Users need access to performance statistics such as FPS, intersection count, and rendering latency.

8. Geometry Export System

Generated wall layouts should be exportable for future reuse and analysis.

Tech Stack

The following technologies were used to build RayCast Vision Lab:

1. React.js

Frontend JavaScript library used to create reusable UI components and manage application state.

2. Vite

Fast development environment and optimized build tool.

3. HTML5 Canvas

Used as the primary rendering engine for drawing rays, walls, visibility polygons, and simulation elements.

4. JavaScript ES6

Used for computational geometry calculations and ray casting algorithms.

5. React Hooks

Used for state management and rendering updates.

6. LocalStorage API

Used to save wall layouts and simulator configurations.

7. CSS3

Used for styling the dashboard, controls, viewport, and telemetry interface.

Component Architecture

Ray Cast Vision Lab follows a component-based React architecture.

```
src/
├── components/
├── SimulatorConsole.jsx
│   └── Control panel
├── CanvasStage.jsx
│   └── Main rendering engine
├── OpticsTelemetryHUD.jsx
│   └── Performance dashboard
├── data/
├── defaultWalls.js
│   └── Default room layout
├── mazeTemplate.jsx
│   └── Maze environment
├── utils/
├── generateRays.jsx
│   └── Ray generation logic
├── getIntersection.jsx
│   └── Collision detection engine
├── App.jsx
│   └── Root state manager
└── main.jsx
    └── Entry point
```

Data Flow

```
User Input
  ↓
SimulatorConsole
  ↓
App.jsx State Manager
  ↓
CanvasStage
  ↓
```

Ray Generation Engine
↓
Intersection Engine
↓
Visibility Polygon
↓
Canvas Rendering
↓
Telemetry HUD

Core Features

1. Ray Sweeper Engine

Generates hundreds of rays from a movable light source.

Users can adjust:

- Field of View
- Ray Density
- Rotation Angle

2. Interactive Wall Builder

Users can:

- Click
- Drag
- Release

to create custom wall boundaries.

The new walls immediately affect ray calculations.

3. Field of Vision Renderer

The simulator joins all closest ray intersections together to create a visibility polygon.

This represents the illuminated area visible to the light source.

4. Rotation Control

Users can rotate the viewing direction.

Examples:

- $0^\circ \rightarrow$ Right
 - $90^\circ \rightarrow$ Down
 - $180^\circ \rightarrow$ Left
 - $270^\circ \rightarrow$ Up
-

5. Local Storage Support

Wall layouts are automatically saved.

Configurations persist even after browser refreshes.

6. JSON Geometry Export

Users can export wall coordinates as JSON files.

Useful for:

- Backup
 - Sharing layouts
 - Future loading systems
-

7. Telemetry HUD

Displays:

- Current Light Position
 - Ray Count
 - Wall Count
 - Intersections
 - FPS
 - Rendering Latency
-

Algorithm Overview

Ray Generation Algorithm

The simulator divides the Field of View into equally spaced angular segments.

Each segment creates a ray:

$\text{Angle Step} = \text{FOV} / \text{Ray Count}$

Ray-Wall Intersection Algorithm

Every ray is checked against every wall segment.

Using parametric line equations:

$$\begin{aligned} 0 &\leq t \leq 1 \\ u &\geq 0 \end{aligned}$$

If valid, an intersection point exists.

The nearest point is selected.

Visibility Polygon Algorithm

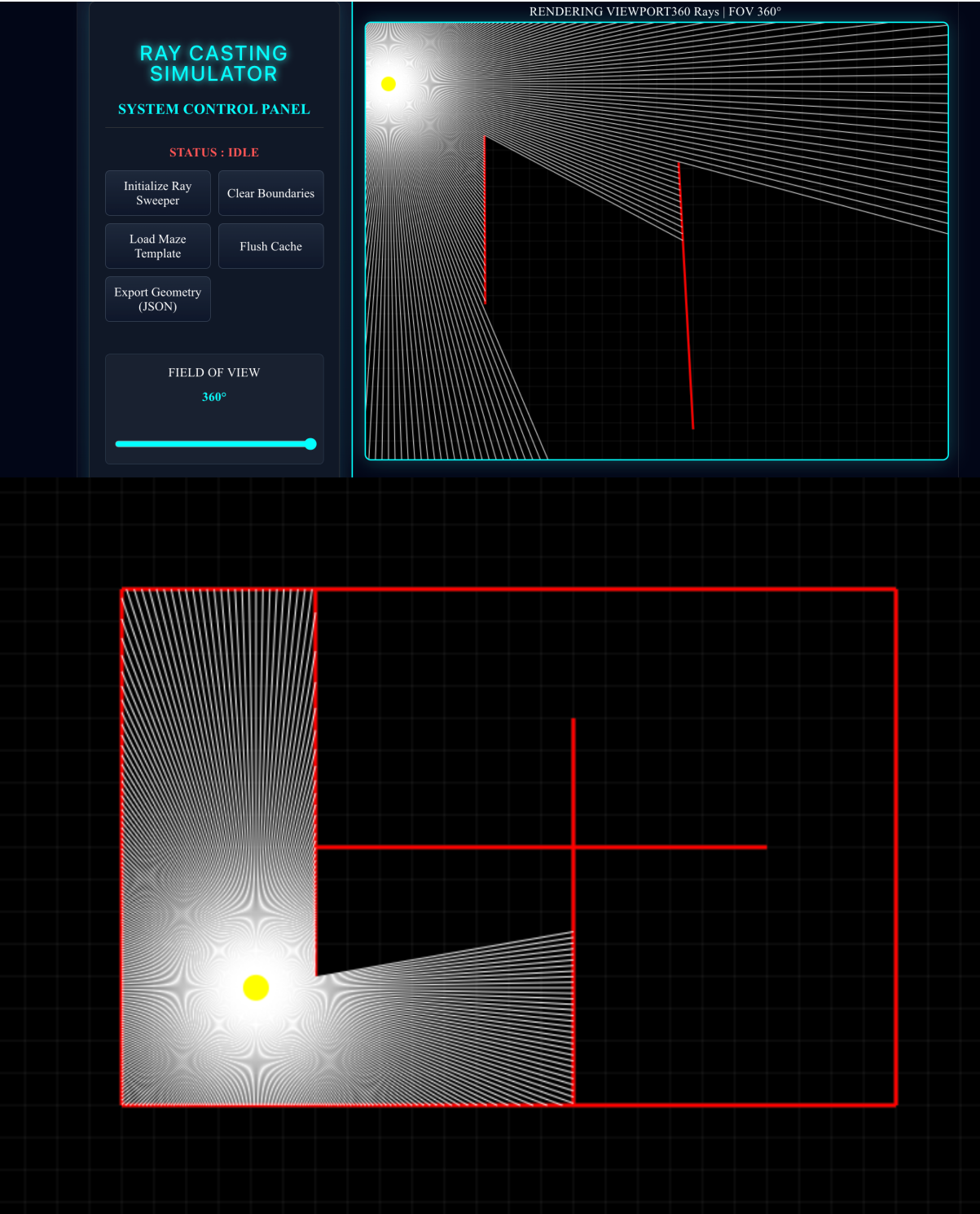
All closest intersection points are stored.

These points are connected to create the visible region.

Rendering Pipeline

```
Generate Rays
  ↓
Check Intersections
  ↓
Find Nearest Hit
  ↓
Build Visibility Polygon
  ↓
Draw Walls
  ↓
Draw Light Source
  ↓
Update Telemetry
```

Screenshots





Conclusion

Ray Cast Vision Lab successfully demonstrates the implementation of real-time ray casting and computational geometry using React.js and HTML5 Canvas.

The project combines:

- Ray Casting
- Vector Mathematics
- Collision Detection
- Visibility Polygon Generation
- Local Storage Persistence
- Performance Monitoring

into a single interactive simulation platform.

All major features of the case study have been successfully implemented, including dynamic field-of-view rendering, wall interaction, rotation control, telemetry monitoring, and environment persistence.

The architecture is modular, scalable, and closely resembles the visibility systems used in robotics, LiDAR scanners, game engines, and computer graphics applications.

Project Links

GitHub Repository

<https://github.com/dharmitsathvara29-max/Ray-Casting-Simulator>

Live Demo

ray-casting-simulator.vercel.app

This format will match the **Documentation(React)-2.pdf** style almost page-for-page and should come out around **10–13 pages after adding screenshots**.